

Manual de referencia de spinpy

Verificación de las métricas morfométricas y de los algoritmos de optimización
de AppFinal_v2

Carlos Gonzalez-Torres

Universidad de Valparaíso

spinpy — núcleo voxel-primero para hueso trabecular

Port a Python de la mitad numérica de AppFinal_V2.m

Documento generado automáticamente a partir de los docstrings del código
fuente. No se edita a mano: se regenera con
`python docs/generar_manual.py`

Licencia MIT. La interfaz gráfica importa PyQt5 (GPL v3 o comercial);
el núcleo spinpy no depende de Qt. Ver LICENSE.

Este manual documenta el software, no valida los resultados obtenidos con él.

La verificación numérica contra soluciones manufacturadas está en
`validar_analitico.py` y en el anexo de validación del proyecto.

Indice

El paquete	4
Generacion: el campo aleatorio gaussiano	4
Funciones	5
Lectura de volúmenes de interés	6
Funciones	6
Morfometría: BV/TV, superficie, MIL, Conn.D, SMI	7
Funciones	7
Espesor local por esferas inscritas	9
Funciones	10
Función de error entre VOI y candidato	10
Funciones	10
Ajuste: búsqueda escalonada y desempate mecánico	11
Funciones	12
Homogeneización elástica periódica	13
Funciones	14
Ensayo de compresión, von Mises y Pistoia	14
Funciones	15
Mallas sólidas: hexaédrica y TET10	17
Funciones	18
Exportadores: Abaqus, APDL, VTU, STL	18
Funciones	19
Lote de VOIs y dispersión del generador	19
Funciones	20
Varianza, ICC, N efectivo y equivalencia TOST	20
Funciones	21
Línea de comandos	22
Funciones	22
Interfaz gráfica	22
Funciones	23
Clase `Trabajador`	23
Clase `DialogoFigura`	24
Clase `DialogoEspesor`	24
Clase `DialogoConvergencia`	24
Clase `DialogoDistribucionFE`	24
Clase `DialogoDispersion`	25
Clase `DialogoVarianza`	25

Clase `Deslizador` 25

Clase `Visor` 25

Ajuste de microestructuras espinodales a volúmenes de hueso trabecular obtenidos por microtomografía.

EN PALABRAS SENCILLAS

Este manual está **generado del código**. Cada apartado reproduce la documentación que acompaña a la función en el fuente, y no una reescritura: en este proyecto los docstrings llevan las decisiones de diseño, los sesgos medidos y las advertencias sobre que números no son citables. Si algo aquí está desactualizado, lo está también en el código, que es donde hay que corregirlo.

El paquete

`__init__.py`

spinpy — núcleo voxel-primero del pipeline de spinodoides, portado a Python.

Cubre la parte de AppFinal_V2.m que va del generador a la morfometría:

`generar_mascara(...)` -> BW (nx, ny, nz) -> morfometria(BW, spacing)

Lee VOIs en VTK legacy (.vtk) y en .mat de MATLAB.

Ajusta un spinodoide a un VOI con la búsqueda escalonada de `fitSpinodoidToVOI` (etapas A/B/C, correcciones G1/G2/K2) mediante `ajustar_spinodoide`.

Homogeneiza elásticamente sobre la rejilla de voxels (homogeneizar).

Ensayo a compresión en X, Y o Z y estima la carga de fallo con el criterio de Pistoia, devolviendo los campos de deformación efectiva y de von Mises (`ensayo_compresion`, `ensayo_compresion_eje`, `ensayo_triaxial`, `criterio_pistoia`).

Lo que NO cubre (sigue solo en MATLAB): exportación FEBio y los otros cuatro optimizadores (Pareto, bayesiano, MOBO y el best-fit rápido con término mecánico).

Generación: el campo aleatorio gaussiano

`grf.py`

grf.py — Generador de spinodoides VOXEL-PRIMERO.

Port a Python del campo aleatorio gaussiano (GRF) de Kumar et al. (2020), tal como lo implementa `spinodoid.m` de GIBBON, con las dos convenciones de muestreo de direcciones de onda que coexisten en la literatura y en el código disponible en este proyecto.

POR QUE VOXEL-PRIMERO

`AppFinal_V2.m` genera la malla triangular con `spinodoid()`, la escala al tamaño físico del VOI y después la voxeliza (`localVoxelizeMesh`) para medirla con `localMorphometryFromVoxels`. El repositorio del profesor (TPMS-Scaffolds-generator) va aún más lejos: malla tetraédrica con CGAL y métricas sobre la malla.

Para un bucle de ajuste ese camino es innecesariamente caro: mallar en cada evaluación domina el coste. El campo GRF ya está definido sobre una rejilla, y la máscara booleana sale de un umbral. Aquí se corta ahí: rejilla -> umbral -> máscara. La morfometría (`morphometry.py`) opera sobre la máscara, que es exactamente la representación que la app usa para comparar VOI y candidato.

LAS DOS CONVENCIONES DE MUESTREO

Los `thetas` son semiangulos de cono alrededor de los ejes x, y, z. Un vector de onda es admisible si cae dentro de alguno de los conos activos. Como repartir `numWaves` vectores entre los conos NO esta fijado por la teoria, y las dos implementaciones disponibles resuelven distinto:

'rechazo' (GIBBON / AppFinal_V2, spinodoid.m:178-206) Se muestrea un candidato isotropo en la esfera y se acepta si cae en CUALQUIER cono. El reparto entre conos resulta proporcional al angulo solido de cada uno: un cono ancho recibe muchas mas ondas que uno estrecho.

'equitativo' (TPMS-Scaffolds-generator, src/TPMS.py:223) `numWaves // n_conos_activos` ondas por cono, cada una muestreada uniformemente dentro de su propio cono. El reparto es 50/50 aunque los conos tengan angulos muy distintos.

Coinciden solo si todos los conos activos tienen el mismo angulo (o hay uno solo, o el caso es isotropo). Con `thetas=(15,45,0)` el rechazo manda ~90% de las ondas al cono de 45 grados y el equitativo un 50%. Producen anisotropias DISTINTAS para los mismos `thetas` nominales; ver `validar_analitico.py`.

Ninguna es "la correcta": son dos definiciones de la misma familia. Lo que no se puede es cruzar resultados entre ambas sin declarar cual se uso.

Convencion de ejes: la mascara devuelta tiene dimension 1 = X, igual que `readVTKVOI` en AppFinal_V2.m y que `voxelize_to_mask` en el repositorio del profesor.

Funciones

```
level_set(rho, erfinv='exacta')
```

Umbral `phi0` tal que $P(\text{GRF} \leq \phi_0) = \rho$ para un GRF $N(0,1)$.

```
phi0 = sqrt(2) * erf^-1(2*rho - 1) [Kumar et al. 2020]
```

Es identico en GIBBON (spinodoid.m:264) y en el repositorio del profesor (TPMS.py:272, salvo por la aproximacion de `erfinv`).

```
wave_directions(num_waves, thetas, R=None, esquema='rechazo', rng=None)
```

Direcciones de onda unitarias, (`num_waves`, 3).

esquema : 'rechazo' (GIBBON/AppFinal_V2) | 'equitativo' (profesor)

```
euler_R(rx_deg, ry_deg, rz_deg)
```

Matriz de rotacion $R = R_z * R_y * R_x$, la convencion de `generateSpinodoid`.

Replica AppFinal_V2.m:1283-1289. Los deslizadores de rotacion de la app solo cubren +-90 grados y no pueden representar todas las orientaciones de un VOI; por eso existe el mecanismo `pendingR` (correccion F4), que deja a los optimizadores forzar una R arbitraria. Aqui R se acepta directamente.

```
campo_grf(resolution, wave_number, num_waves, thetas, R=None, esquema='rechazo', seed=None, domain_size=1.0, motor='numpy', precision='f64', bloque=None)
```

Evalua el GRF sobre la rejilla del dominio.

Replica spinodoid.m:212-232: `rejilla = linspace(0, domainSize, resolution)` en los tres ejes, `ndgrid -> dimension 1 = X` $\text{GRF}(x) = \sum_i \sqrt{2/N} \cos(\text{waveNumber} <n_i, x> + \gamma_i)$

Las fases se muestrean uniformes en $[0, 2\pi]$. GIBBON usa `rand_angle`, que pese a su construccion rebuscada es uniforme en $[0, 2\pi]$ — se comprobo analiticamente — de modo que `rng.uniform` es equivalente en distribucion.

El campo se evalua por bloques de PUNTOS con un producto matricial (ver `_evaluar_campo`), no por bloques de ondas con difusion. Materializar (`res^3`, `num_waves`) de una vez ocuparia ~1.5 GB a $64^3 \times 700$.

precision : 'f64' (por defecto, exacta) | 'f32' (~3x, ver `_evaluar_campo`)

El orden de consumo del generador aleatorio —primero las direcciones, despues las fases— se mantiene identico al de la version anterior, de modo que una misma semilla sigue dando la misma realizacion.

Devuelve (GRF, dirs, fases) con GRF de forma (res, res, res), dim 1 = X.

```
generar_mascara(resolution, wave_number, num_waves, thetas, rho, R=None, esquema='rechazo',
seed=None, domain_size=1.0, erfinv='exacta', mayor_componente=False, motor='numpy',
precision='f64')
```

Mascara booleana solida (True = hueso), forma (res, res, res), dim 1 = X.

Fase solida = {GRF <= phi0}, la misma convencion que `spinodoid.m` usa al llamar a `isosurface(..., levelset)` con `isocaps 'enclose', 'below'`.

AVISO sobre el codigo del profesor: alli `implicit_func` devuelve GRF + phi0 y la fase solida sale de `pgal.Difference(cuboid, tpms)`, es decir de DOS inversiones de signo encadenadas. Si se toma su `implicit_func` y se umbraliza directamente en `phi < 0` se obtiene la fase PORO, no la osea. Aqui la convencion es explicita y directa.

`mayor_componente` : si True, conserva solo la mayor componente conexa por caras (conectividad 6), analogo a `localLargestComponent6`. La app filtra la mayor componente de SUPERFICIE via `tesgroup`, que no es exactamente lo mismo; por eso el valor por defecto es False y el filtro se declara explicitamente cuando se quiere.

`_5` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Lectura de volúmenes de interes

`io.py`

`io.py` — Lectura de VOIs en VTK legacy ASCII (STRUCTURED_POINTS).

Port de `readVTKVOI` (AppFinal_V2.m:5147-5205) y de la binarizacion que aplica `computeVOIMetrics` (AppFinal_V2.m:5229).

CONVENCION DE EJES — la invariante que hay que respetar

El formato VTK escribe el indice X como el que varia mas rapido, asi que `reshape(data, [nx ny nz])` de MATLAB (orden por columnas) deja la DIMENSION 1 = X. Es lo que documenta la correccion del bug de ejes X/Y intercambiados en `computeVOIMetrics`: la version anterior asignaba la dimension 1 a Y y con voxeles anisotropos —lo normal en micro-CT— deformaba la geometria medida y devolvía una direccion principal erronea, que es la que fija la rotacion aplicada al spinodoide.

En numpy el equivalente exacto de ese reshape es `order='F'`. Usar el orden C por defecto transpone el volumen en silencio.

Funciones

```
leer_vtk_voi(ruta, binarizar=True, umbral=0)
```

Lee un VOI de un VTK legacy ASCII STRUCTURED_POINTS.

Devuelve (VOI, spacing) con VOI de forma (nx, ny, nz), dimension 1 = X.

`binarizar` : aplica `VOI > umbral`, igual que `computeVOIMetrics`, que hace `if ~islogical(VOI), VOI = VOI > 0; end`. Los VOIs de H4 vienen como `unsigned_char` con valores 0 y 255.

```
leer_mat_voi(ruta, binarizar=True, umbral=0)
```

Lee un VOI guardado como .mat de MATLAB (variables `VOI` y `spacing`).

Es el formato del subconjunto porcino publico (Koria, Mengoni & Brockett 2020, Research Data Leeds, CC BY 4.0). El array ya viene con dimension 1 = X porque MATLAB lo guardo asi; scipy.io.loadmat lo devuelve con la misma disposicion, sin necesidad de transponer.

```
leer_voi(ruta, **kw)
```

Lee un VOI eligiendo el lector por la extension (.vtk o .mat).

Morfometria: BV/TV, superficie, MIL, Conn.D, SMI

```
morphometry.py
```

morphometry.py — Port de la morfometria de AppFinal_V2.m.

Traduccion linea a linea de: localMorphometryFromVoxels (AppFinal_V2.m:5890-5996) localMILTensor (AppFinal_V2.m:5733-5866) localLoadBearingFraction (AppFinal_V2.m:6012-6030) localLargestComponent6 (AppFinal_V2.m:6502-6572)

Se respetan las invariantes documentadas en CLAUDE.md, que son correcciones ya pagadas en el codigo MATLAB y que un port ingenuo reintroduce en silencio:

- $Tb.Th = 2 \cdot BV/BS$ (modelo de placas de Parfitt, como CTAn/Scanco). NO $4 \cdot BV/BS$, que es el modelo de barras y duplica el valor.
- DA sale del TENSOR MIL (Harrigan & Mann 1984), nunca de la covarianza de la nube de puntos. El MIL tiene un suelo de ruido de $DA \sim 1.07$: por debajo de esa diferencia no hay significado.
- Las estructuras laminares degeneran el ajuste del elipsoide; se acota el autovalor y se marca el resultado como COTA INFERIOR (MIL_clamped).
- La superficie se mide con marching cubes sobre la mascara binaria, igual que el VOI real, para que el sesgo del metodo se cancele al restar.

CONVENCION DE EJES — el punto donde mas facil es equivocarse

MATLAB llama `isosurface(permute(BW,[2 1 3]), 0.5)` porque `isosurface` devuelve los vertices en convencion meshgrid (columna, fila, pagina); la permutacion previa deja $V(:,1) = X$. Es el bug B1/B2 del historial del proyecto.

En Python NO hace falta permutar: `skimage.measure.marching_cubes` devuelve los vertices en orden de indice del array, asi que con BW de forma (nx, ny, nz) los vertices ya salen como (X, Y, Z). Se usa `method='lorensen'` (marching cubes clasico) por ser el mas proximo al de MATLAB; el 'lewiner' por defecto resuelve las ambigüedades de otra manera y da areas ligeramente distintas.

Funciones

```
mayor_componente_6(BW)
```

Mayor componente conexa por caras. Port de localLargestComponent6.

```
fraccion_portante(BW)
```

Fraccion de material en componentes que unen las dos caras en Z.

Port de localLoadBearingFraction (K3). Vale 1 cuando todo el material forma parte de caminos que atraviesan la probeta y baja al aparecer islas y munones ciegos. Se mide, no se optimiza (peso 0 en el error).

```
area_superficie(BW, spacing, metodo='lorensen')
```

Area de la isosuperficie 0.5 de la mascara, en unidades fisicas.

Equivale a las lineas 5942-5950 de AppFinal_V2.m. Devuelve 0.0 si no hay interfaz (mascara vacia o llena).

```
tensor_mil(BW, spacing, n_dirs=61, n_lines=32, min_chord_frac=0.35)
```

Mean Intercept Length y tensor de fabrica. Port de localMILTensor.

Direcciones cuasi-uniformes en el hemisferio por espiral de Fibonacci; para cada direccion se lanza una rejilla de n_lines^2 rectas paralelas, se acumula longitud osea y numero de transicciones solido->poro, y

$MIL(d) = longitud_osea_total / (0.5 * n_transiciones)$

El factor 0.5 es la correccion F5: totalInt cuenta la entrada Y la salida de cada cuerda, es decir 2 por cuerda, y el MIL estandar (CTAn/Scanco) divide por el numero de cuerdas. Sin el factor se devuelve la mitad del valor. El DA no se ve afectado por ser un cociente, pero el MIL expuesto si.

Despues se ajusta por minimos cuadrados el elipsoide $1/MIL^2 = n' M n$ y se diagonaliza. El autovalor MINIMO corresponde al MIL MAXIMO, es decir a la direccion principal de la estructura.

Devuelve un dict con las mismas claves que el struct fab de MATLAB.

```
conectividad(BW, spacing)
```

Densidad de conectividad (Conn.D), Odgaard & Gundersen 1993.

$Conn = 1 - \chi$ $Conn.D = Conn / TV [1/mm^3]$

siendo χ la caracteristica de Euler de la fase solida. Cuenta las conexiones REDUNDANTES de la red: cuantas trabeculas se podrian cortar antes de partirla en dos.

POR QUE IMPORTA Y POR QUE NO LA SUSTITUYE BV/TV. En perdida osea las trabeculas no solo adelgazan: se desconectan. Dos estructuras con el mismo BV/TV y el mismo Tb.Th pueden tener redes completamente distintas, y la resistencia depende de la red, no solo de cuanto material hay. Es la metrica que separa "hueso mas fino" de "hueso roto".

Se usa conectividad 26 para el solido, que es la convencion de Odgaard (emparejada con 6 para el fondo) y la que aplican CTAn y Scanco.

```
indice_smi(BW, spacing, dr_rel=0.05, metodo='lorensen')
```

Structure Model Index (Hildebrand & Rueggsegger 1997).

$SMI = 6 BV (dBS/dr) / BS^2$

Mide si la estructura se parece mas a placas o a barras dilatandola un poco y viendo como cambia la superficie: en una placa el area apenas cambia ($SMI \sim 0$), en una barra crece linealmente ($SMI \sim 3$) y en una esfera crece mas deprisa ($SMI \sim 4$).

LA CRITICA, QUE HAY QUE REPORTAR JUNTO AL NUMERO. Salmon et al. (2015) mostraron que el SMI esta confundido por la CONCAVIDAD: las superficies concavas aportan area negativa al dilatar, asi que en hueso denso el indice sale negativo y deja de tener la interpretacion placa-barra. No es un artefacto de calculo, es una limitacion de la definicion. Se incluye porque se sigue pidiendo en la literatura, no porque sea fiable a BV/TV alto.

La dilatacion se hace desplazando los vertices de la isosuperficie a lo largo de sus normales, no con morfologia sobre voxels: un paso de un voxel entero seria demasiado grosero para una derivada.

SESGO DE DISCRETIZACION, MEDIDO. Sobre geometrias de respuesta conocida:

esfera +3.45 (exacto 4) -13.8% cilindro +2.68 (exacto 3) -10.8% losa 0.00 (exacto 0) exacto

El deficit no es un fallo del calculo: marching cubes sobreestima BS un 8.9% sobre datos binarios y el SMI va como $1/BS^2$, asi que cabe esperar -15.7%. La losa sale exacta porque su dBS/dr es cero y el sesgo de BS no tiene por donde entrar. Toda implementacion sobre voxels arrastra esto.

```
morfometria(BW, spacing, do_mil=True, modelo_tbth='plate', metodo_mc='lorensen', extra=False)
```

Port de localMorphometryFromVoxels.

Es el UNICO sitio donde se calculan BV/TV, BS/BV, Tb.Th, Tb.Sp, Tb.N y DA, igual que en la app: cualquier metrica nueva debe pasar por aqui, porque el sesgo del metodo solo se cancela si VOI y candidato recorren el mismo camino (correccion C4).

spacing en mm/voxel. Devuelve un dict con las claves del struct de MATLAB.

Espesor local por esferas inscritas

`espesor.py`

`espesor.py` — Campo de espesor local (Tb.Th punto a punto).

Metodo de Hildebrand y Ruegsegger (1997), que es el que usan CTAn y Scanco: el espesor local en un punto es el DIAMETRO DE LA MAYOR ESFERA que contiene ese punto y cabe entera dentro de la estructura.

$$Th(y) = 2 * \max \{ r(x) : |y - x| \leq r(x) \}$$

siendo $r(x)$ la distancia de x al fondo, es decir el radio de la mayor esfera CENTRADA en x que cabe.

POR QUE NO BASTA LA TRANSFORMADA DE DISTANCIA

Es tentador colorear directamente por la distancia al fondo y llamarlo espesor. No lo es: $r(x)$ solo da el espesor en el eje medial, y cae a cero al acercarse a la superficie. Como la superficie es justo lo que se ve al renderizar, colorear por $r(x)$ mostraria una estructura uniformemente delgada por fuera, con el valor bueno escondido en el interior. El maximo sobre las esferas que CONTIENEN el punto —no las centradas en el— es lo que reparte el espesor del eje medial a todo el grosor de la trabecula.

SESGO DE DISCRETIZACION — MEDIDO, Y GRANDE A ESTA ESCALA

El metodo mide distancias entre CENTROS de voxel, asi que la frontera efectiva del objeto digitalizado cae medio voxel por dentro y el espesor sale corto. El sesgo depende de cuantos voxeles cruzan la estructura (cilindro, medido):

diametro (voxeles) 2 3 4 6 10 20 error 0.0% -33% -29% -5.7% -11% -5.7%

Una losa plana sale EXACTA (0.00%), porque no hay curvatura que discretizar.

Esto importa mucho aqui: los VOIs equinos tienen Tb.Th ~0.17 mm con voxel de 0.0515 mm, es decir unos 3.3 voxeles de diametro, justo en la zona donde el sesgo ronda el 30%. LOS VALORES ABSOLUTOS DE ESTE CAMPO NO SON CITABLES COMO Tb.Th.

Para lo que si sirve:

- ver DONDE la estructura es gruesa o fina, que es informacion relativa y el sesgo es sistematico;
- comparar VOI y spinodoide medidos con el MISMO voxel, porque entonces el sesgo afecta por igual a los dos y se cancela en gran medida al restar —el mismo argumento de la correccion C4.

Para un numero citable esta el Tb.Th de Parfitt que devuelve `morfometria`.

DIFERENCIA CON EL Tb.Th DE LA MORFOMETRIA

`morfometria` devuelve el Tb.Th de Parfitt, $2*BV/BS$, que es un MODELO DE PLACAS paralelas: un unico numero por VOI, obtenido de un cociente global. Este modulo da un campo, y su media NO tiene por que coincidir con aquel valor —son dos definiciones distintas, y las dos son estandar. La de Parfitt es la que reportan CTAn y Scanco por defecto y la que usa el ajuste; esta sirve para VER donde la estructura es gruesa o fina.

Funciones

```
espesor_local(BW, spacing=1.0, n_radios=None)
```

Campo de espesor local, en las mismas unidades que `spacing`.

Devuelve un array del tamaño de `BW` con el espesor en cada voxel sólido y cero en el fondo.

El algoritmo recorre los radios de mayor a menor. Para cada radio r toma los centros que admiten una esfera de al menos ese radio y marca todo lo que queda a distancia $\leq r$ de alguno; lo que se marca por primera vez tiene espesor $2r$. Al ir de mayor a menor, cada voxel se queda con el mayor radio que lo cubre, que es la definición.

`n_radios` limita cuantos niveles se prueban (uno por nivel cuesta una transformada de distancia). Con `None` se usan todos los radios distintos que aparecen, redondeados a la rejilla.

```
estadisticas(esp, BW=None)
```

Resumen del campo: media, mediana, desviación y percentiles.

```
muestrear_en_puntos(esp, spacing, puntos, origen=None)
```

Valor del campo en coordenadas arbitrarias, por vecino más próximo.

Los vértices de la isosuperficie caen ENTRE voxels, y con frecuencia del lado del fondo, donde el espesor vale cero. Colorear con eso pintaría toda la piel de azul. Se dilata antes el campo con un máximo $3 \times 3 \times 3$ para que cada vértice recoja el valor del sólido contiguo.

Funcion de error entre VOI y candidato

```
error.py
```

`error.py` — Función de error morfométrico y criterios de fábrica.

Port de: `computeMorphometricError` (AppFinal_V2.m:3725-3785) `localFabricCos` (AppFinal_V2.m:6051) `localFabricAlignWeight` (AppFinal_V2.m:6076) `localTieneEje` (AppFinal_V2.m:6035)

INVARIANTE C6 — LA RAZÓN DE SER DE ESTA FUNCIÓN

La versión anterior de la app sumaba los términos disponibles y saltaba EN SILENCIO los que salieran NaN. Consecuencia: un candidato en el que alguna métrica no se podía calcular acumulaba MENOS error que uno en el que sí se calculaba, y por tanto parecía mejor. El optimizador quedaba sesgado justamente hacia las geometrías degeneradas, que son las que más métricas hacen fallar.

La corrección es dividir por la suma de pesos EFECTIVAMENTE usados, de modo que el error es un promedio ponderado y no depende de cuantos términos sobrevivan. Si se rompe esto, el sesgo vuelve sin dar ningún síntoma.

TERMINO MECANICO (J1)

La app puede añadir `Ez_rel` y `Ez_Ex`, que requieren homogeneización elástica. El port aún no la cubre, así que esos campos no existen y la función se comporta como la versión sin término mecánico, bit a bit — que es exactamente lo que hace MATLAB cuando el ajuste mecánico está desactivado. Los pesos ya están escritos abajo para cuando se porte.

Funciones

```
error_morfometrico(m_voi, m_spin, peso_mecanico=0.0)
```

Error morfométrico ponderado entre VOI y candidato.

Devuelve (`err`, `n_usados`). `err` es un promedio ponderado de diferencias relativas al cuadrado; vale PENALIZACIÓN si no hubo ningún término utilizable.

`peso_mecanico` : equivalente de `localMechFitWeight()`. Escala los pesos de `Ez_rel` y `Ez_Ex`, que salen de la homogeneización periódica (`spinpy.fit.metrics_mecanicas`). Si se pone > 0 pero esos campos no están en los `dicts`, simplemente no se usan y el error se renormaliza por el peso realmente aplicado (invariante C6): un término ausente no puede contar como un término con error cero.

En el ajuste no se activa dentro de la búsqueda sino en una etapa de desempate entre finalistas — homogeneizar las ~53 evaluaciones sería entre tres y cuatro órdenes de magnitud más caro. Ver `ajustar_spinodoide`.

`tiene_eje(m)`

Port de `localTieneEje`: ¿el struct trae una dirección principal usable?

`fabric_cos(m_a, m_b)`

Coseno del ángulo entre direcciones principales, en valor absoluto.

Se compara como EJE y no como vector: el signo de un autovector es arbitrario, de ahí el valor absoluto.

`peso_alineacion(m_voi, da_min=1.15, da_pleno=1.37, w_max=2.0)`

Port de `localFabricAlignWeight`.

Cuanto vale desempatar por orientación en ESTE VOI. Devuelve 0 —es decir, no desempatar— cuando la dirección principal no significa nada:

- DA por debajo de 1.15: por debajo del suelo de ruido del MIL (~1.07) la dirección principal es ruido, no estructura.
- MIL acotado: en estructuras laminares el elipsoide degenera y el DA es solo una cota inferior.

Entre 1.15 y 1.37 el peso sube linealmente hasta el máximo.

`_1` función(es) interna(s) de este módulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz pública._

Ajuste: búsqueda escalonada y desempate mecánico

`fit.py`

`fit.py` — Ajuste de un spinodoide a un VOI por búsqueda escalonada.

Port de `fitSpinodoidToVOI` (`AppFinal_V2.m:3407-3720`), incluidas las correcciones G1, G2 y K2, que son la parte más reciente del trabajo sobre la app y la que motivó la versión V2.

LAS TRES ETAPAS

A. densidad x número de onda (ángulos fijos en el valor de partida) B. presets de ángulos cónicos (con los mejores parámetros de A) C. refinado de densidad y número de ondas (solo en modo completo)

Las correcciones que NO se pueden perder al portar:

G2 — el número de onda se recorre en TODO su rango útil [8, 25] en el modo completo, con independencia de donde estuviera el deslizador. La versión anterior usaba una ventana [base-5, base, base+5]; con el valor por defecto (15) eso daba [10 15 20] y NUNCA alcanzaba 25, de modo que en trabéculas muy finas el ajuste sobreestimaba $T_b.T_h$ en torno al 43 %.

G1 — los ángulos cónicos se BARREN en la etapa B. Antes quedaban fijos en el valor de los deslizadores y el spinodoide salía casi isotropo (DA ~1.1-1.2), incapaz de alcanzar el DA 1.4-1.5 del hueso proximal equino.

K2 — la lista de presets cubre las PERMUTACIONES, no solo un octante. θ no es simétrico: el ángulo grande marca el eje en torno al cual se concentran los vectores de onda, y por tanto el eje BLANDO. Sin las permutaciones solo se podía orientar la estructura de una manera. Además se desempata por orientación entre los presets cuyo error

ya esta dentro del 5 % del mejor (ver `_desempatar_theta`).

POR QUE LA ORIENTACION NO ES UN TERMINO DEL ERROR

Se probó y se descartó: sobre los tres VOI de H4 con 5 replicas mejoraba la rigidez pero subia el error de las siete metricas de 0.00726 a 0.00990 (+36 %). Un sumando nuevo compra su objetivo con el de los demas y no hay forma de acotar el intercambio desde dentro de la suma. Aqui la orientacion es criterio SECUNDARIO entre candidatos ya buenos: la morfometria queda acotada por construccion y la formula del error no se toca.

Funciones

`metricas_mecanicas(BW, spacing, res_mec=16, E_s=E_S_DEF, nu_s=NU_S_DEF)`

`Ez_rel` y `Ez_Ex` de una estructura, por homogeneizacion periodica.

Son los dos campos mecanicos que `error_morfometrico` sabe usar cuando se le da `peso_mecanico > 0`: la rigidez axial normalizada y la anisotropia elastica.

La estructura se REMUESTREA a `res_mec` antes de homogeneizar porque el coste crece con el cubo del lado: a 16^3 son ~6 s y a 32^3 ~99 s. En un desempate entre finalistas eso es la diferencia entre medio minuto y media hora.

Devuelve {} si la homogeneizacion falla, y no ceros: `error_morfometrico` salta los campos ausentes y renormaliza por el peso realmente usado (invariante C6), mientras que un cero fingiria una estructura sin rigidez y arrastraria el ajuste hacia geometrias degeneradas.

`Ez_rel` DEPENDE FUERTEMENTE DE `res_mec`, y hay que contar con ello. Medido sobre el mismo VOI sintetico (48^3 , BV/TV 0.345): `Ez_rel` = 0.108 a 12^3 y 0.064 a 16^3 . No es un error de calculo sino el mismo fenomeno que hace no convergente a `E_app` en el ensayo de compresion —al remuestrear cambia que trabeculas quedan conectadas— y por eso el numero **no es citable como rigidez del espécimen**.

Para lo que si sirve es para comparar: en el ajuste, VOI y candidatos se homogeneizan todos al MISMO `res_mec`, de modo que el sesgo actua sobre los dos lados y se cancela en buena parte al restar. Es exactamente el argumento de la correccion C4, y es la unica razon por la que un `res_mec` barato es aceptable aqui y no lo seria para reportar un modulo.

`dir_a_euler(dir_vec)`

Port de `dirVectorToEulerApprox` (AppFinal_V2.m:5414).

Angulos de Euler que llevan el eje z sobre `dir_vec`. Se fuerza la componente z positiva porque la direccion principal es un EJE: su signo es arbitrario y quedarse con el hemisferio superior evita rotaciones de 180 grados que no cambian la geometria.

`longitud_caracteristica(voi_shape, spacing)`

Port de `localCharacteristicLength`: lado fisico medio del VOI, en mm.

Correccion F1: el spinodoide se genera en un cubo unidad adimensional. Si no se escala al tamano fisico del VOI antes de medirlo, Tb.Th, Tb.Sp y BS/BV salen en unidades distintas y restarlas no significa nada.

`resolucion_comparacion(voi_shape)`

Port de `localVoxelCountForComparison`: $24 \leq n \leq 96$, tomado del VOI.

`fraccion_poro_conexa(BW)`

Fraccion del poro que pertenece a su mayor componente conexa (caras).

Mide si la estructura sigue siendo BICONTINUA. En hueso trabecular la medula forma una red conectada; cuando este valor cae, la estructura ha pasado a ser un solido con poros aislados y deja de ser trabecula, por bien que coincida el BV/TV.

```
ajustar_spinodoide(VOI, spacing, modo='completo', m_voi=None, esquema='rechazo',
precision='f32', seed=SEMILLA_POR_DEFEECTO, num_waves=700, resolucion=None, progreso=None,
rho_max=RHO_MAX, peso_mecanico=0.0, res_mec=16, n_finalistas=5, E_s=E_S_DEF, nu_s=NU_S_DEF)
```

Busca los parametros de spinodoide que mejor reproducen el VOI.

modo : 'completo' (etapas A+B+C, 14 presets) | 'rapido' (A+B, 4 presets)

peso_mecanico : si es > 0 se anade una ETAPA D que reordena los `n_finalistas` mejores candidatos incluyendo `Ez_rel` y `Ez_Ex`, obtenidos por homogeneizacion a `res_mec`.

POR QUE UNA ETAPA APARTE Y NO DENTRO DE LA BUSQUEDA. Homogeneizar es entre tres y cuatro ordenes de magnitud mas caro que medir morfometria: las ~53 evaluaciones del modo completo pasarian de segundos a horas. La etapa D paga la homogeneizacion solo donde puede cambiar la decision —entre candidatos que ya son buenos morfometricamente— y son `n_finalistas+1` homogeneizaciones en total, contando la del VOI.

LO QUE ESTO NO ES. No es una busqueda con el termino mecanico dentro: si el optimo mecanico esta en una region que la busqueda morfometrica descarto pronto, la etapa D no lo encuentra. Es un desempate informado, y `traza_mecanica` deja ver cuanto reordeno para que se pueda juzgar si el desempate estaba decidiendo algo o solo confirmando.

precision : 'f32' por defecto en la busqueda. Son ~50 evaluaciones y la diferencia frente a f64 no cambia ningun voxel en los casos medidos (ver `spinpy.grf._evaluar_campo`). El candidato ganador se REMIDE al final en f64, de modo que las metricas que se reportan son exactas.

Devuelve un dict con los parametros ganadores, sus metricas y la traza.

`_1` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Homogeneizacion elastica periodica

`elastic.py`

`elastic.py` — Homogeneizacion numerica periodica sobre la rejilla de voxeles.

Port de: `localHex8Ke` (AppFinal_V2.m:6121-6167) `localHomogenizeVoxel` (AppFinal_V2.m:6192-6430) `localEngineeringConstants`(AppFinal_V2.m:6452-...)

Resuelve el problema de celda unidad de verdad —elementos hexaedricos sobre la rejilla de voxeles, condiciones de contorno periodicas, seis casos de deformacion macroscopica unitaria— y obtiene

$$C_{ij} = (1/V) * \sum_e (u0_i - chi_i)' ke (u0_j - chi_j)$$

siendo `chi` el campo corrector. Referencia: Andreassen & Andreasen, Comput. Mater. Sci. 83 (2014) 488-495.

SUSTITUYE al tensor heuristico que la app tenia antes ($E_{eff} = E_s * \rho^2$ con un reparto lineal segun los angulos y un factor 0.9 sin justificacion), en el que la rigidez no derivaba de la geometria en ningun momento: dos estructuras con la misma densidad daban el mismo tensor aunque su topologia fuera distinta.

CONVENCIONES QUE NO SE PUEDEN CAMBIAR

-
- Voigt [xx yy zz yz xz xy] con deformaciones angulares de INGENIERIA, en todo el archivo y en todo el proyecto.
 - El vacio se modela con rigidez $1e-6 * E_s$ en lugar de eliminarse: evita que islas de material desconectadas hagan singular la matriz, a costa de un sesgo despreciable frente al solido.
 - La rejilla de NODOS tiene el mismo tamano que la de elementos: el nodo $nx+1$ es, por periodicidad, el nodo 1.

- Los arrays se recorren en orden Fortran (columna primero) para que el índice de elemento coincida con el de MATLAB.

Funciones

`hex8_ke(dx, dy, dz, E, nu)`

Rigidez de un hexaedro trilineal de 8 nodos, Gauss 2x2x2.

Se calcula numericamente en lugar de usar una matriz precomputada: mas largo, pero verificable y sin riesgo de erratas de transcripcion.

`homogeneizar(BW, E_s=1.0, nu_s=0.3, vox_size=1.0, tol=1e-08, escala_vacio=VOID_SCALE, verbose=False)`

Tensor de rigidez homogeneizado C_h (6x6) en orden de Voigt.

Devuelve (Ch, info). Si algo impide resolver, Ch es NaN e `info['msg']` explica por que — nunca se devuelve un tensor silenciosamente equivocado.

`escala_vacio` : rigidez de la fase `False` relativa a la solida. El valor por defecto $1e-6$ es el de la app y representa vacio. Se expone como parametro porque permite VERIFICAR la funcion contra el promedio de Backus con dos fases reales: con vacio a $1e-6$ casi todas las entradas del tensor exacto valen ~ 0 y la comparacion no distingue una implementacion correcta de una equivocada.

`remuestrear_bw(BW, spacing, n_max)`

Reduce la mascara a `n_max` voxeles por eje. Port de `localResampleBW`.

Submuestreo por vecino mas proximo, no por promediado: la mascara debe seguir siendo binaria. El `spacing` se corrige por el factor real alcanzado, no por el pedido, porque `round` sobre `linspace` no da exactamente `n_max`.

El coste de la homogeneizacion crece con el CUBO del lado, asi que este recorte no es cosmetico: es lo que hace viable calcular un tensor sobre un VOI de 97^3 .

`constantes_ingenieria(C)`

Constantes de ingenieria a partir de C (6x6), via $S = \text{inv}(C)$.

Orden de Voigt `[xx yy zz yz xz xy]` con deformaciones angulares de ingenieria, de modo que $S[3,3] = 1/G_{yz}$ directamente.

`backus_laminado(fracciones, E, nu)`

Tensor EXACTO de un laminado de capas isotropas normales a z .

Promedio de Backus (1962). Es la solucion analitica contra la que se verifica `homogeneizar`: para un apilado de capas planas la solucion de elementos finitos debe reproducirla, porque el campo corrector es constante dentro de cada capa y el hexaedro trilineal lo representa exactamente.

`fracciones` : fracciones de volumen de cada capa (suman 1) `E`, `nu` : modulos y coeficientes de Poisson de cada capa
`_3` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Ensayo de compresion, von Mises y Pistoia

`resistencia.py`

`resistencia.py` — Ensayo de compresion axial y carga de fallo aparente.

Cubre lo que a la rigidez le falta: para biomecanica, saber que un hueso es rigido es media historia; la pregunta que suele importar es a que carga rompe.

CRITERIO DE PISTOIA

Pistoia et al. (Bone 30:842, 2002). Sobre una UNICA resolucion LINEAL se busca el factor por el que hay que escalar la carga para que un porcentaje del tejido supere una deformacion critica:

el hueso falla cuando el 2% del volumen oseo supera el 0.7% de deformacion efectiva

Como el problema es lineal, la deformacion escala con la carga y el factor sale de una division: no hace falta analisis no lineal ni iterar. El criterio se valido originalmente contra ensayos de radio distal humano y predice la carga de rotura mucho mejor que la densidad sola.

La deformacion efectiva es la definicion energetica que usa esa literatura:

$\text{eps_eff} = \sqrt{2 * U / E_s}$ con U la densidad de energia de deformacion

Para un estado uniaxial se reduce a $|\text{eps_zz}|$, lo que permite comprobarla contra una solucion exacta.

LOS DOS PARAMETROS SON CONVENCIONES, NO CONSTANTES FISICAS

El 2% y el 0.7% se calibraron en radio distal humano a una resolucion concreta. No son universales: dependen de la especie, del sitio y —esto importa— de la RESOLUCION de la malla, porque el percentil de una cola depende de cuantos elementos haya. Se exponen como argumentos y se reportan siempre junto al resultado, en lugar de esconderlos como constantes.

CONDICIONES DE CONTORNO

'empotrado' base totalmente fija: reproduce un ensayo con friccion entre probeta y platos, introduce coaccion lateral y rigidiza. 'deslizante' $u_z = 0$ en la base y solo lo minimo para eliminar los movimientos de solido rigido: compresion sin friccion, estado uniaxial.

La eleccion cambia E_{app} y la carga de fallo, asi que hay que declararla.

DOS CAMPOS DE SALIDA, NO UNO

Del mismo campo de tensiones salen `campo_eps_eff` (deformacion efectiva de Pistoia, energetica) y `campo_vm` (von Mises, solo la parte desviadora). No son el mismo mapa reescalado: coinciden salvo un factor en estado uniaxial y se separan en los nudos trabeculares, donde el estado es triaxial. Uno predice la carga aparente de fallo y el otro localiza donde plastificaria el tejido.

TRES EJES

`ensayo_compresion_eje` corre el mismo ensayo en X, Y o Z permutando la estructura, y `ensayo_triaxial` los tres de una vez. El cociente E_{max}/E_{min} que sale de ahi es una anisotropia MECANICA, contrastable con $E_x/E_y/E_z$ de la homogeneizacion periodica y distinta del DA del MIL, que es geometrico.

Funciones

```
ensayo_compresion(BW, spacing, E_s=E_S_DEF, nu_s=NU_S_DEF, sigma0=SIGMA0_DEF,  
apoyo='deslizante', escala_vacio=None, tol=1e-08, solo_portante=True, progreso=None)
```

Compresion axial en z. Devuelve campos y magnitudes aparentes.

La tension aparente `sigma0` se define sobre la seccion BRUTA del VOI (lado x lado), no sobre el area osea. Es la convencion de la app y la que hace comparables los resultados con la morfometria: aplicarla sobre el area real daria una tension mucho mayor y dependiente del BV/TV.

EL VACIO NO SE MALLA — y por que importa

`homogeneizar` rellena los poros con un material de rigidez $1e-6 * E_s$ para que la celda periodica nunca sea singular. Aqui eso NO funciona: medido sobre el VOI proximal de H4 a 32^3 (BV/TV 0.28), el solver se quedaba en un residuo de $3.3e-1$ —sin converger en absoluto— porque el contraste de $1e6$ entre hueso y relleno arruina el condicionamiento en una estructura poco densa.

La salida es la de la practica habitual en micro-elementos finitos (Van Rietbergen): mallar SOLO el hueso y reenumerar los nodos. El sistema queda bien condicionado mientras el hueso este conectado, que es justo lo que garantiza `solo_portante`. Ademas es mas rapido: a BV/TV 0.28 hay un 72% menos de elementos.

`escala_vacio` se conserva por compatibilidad; si se le da un valor, se vuelve al modelo antiguo con relleno blando.

`criterio_pistoia(res, frac=FRAC_CRITICA, eps_crit=EPS_CRITICA)`

Carga de fallo aparente a partir de un ensayo lineal ya resuelto.

Devuelve el factor por el que hay que escalar la carga para que la fraccion `frac` del volumen ose alcance `eps_crit` de deformacion efectiva, y la tension y carga de fallo que resultan.

Como el problema es LINEAL, eps escala con la carga y el factor es una division. Toda la maquinaria no lineal que uno esperaria aqui sobra: esa es justamente la gracia del criterio.

`ensayo_compresion_eje(BW, spacing, eje=2, **kw)`

`ensayo_compresion` a lo largo de X, Y o Z (`eje = 0, 1 o 2`).

El ensayo esta escrito para comprimir en z —el filtro portante, la cara cargada, el apoyo y `E_app` miran todos a la tercera dimension—. En vez de duplicar esa logica con indices parametrizados, que es donde se cuelan los errores que no dan error, se PERMUTA la estructura para que el eje pedido ocupe la posicion z, se resuelve el mismo codigo de siempre y se deshace la permutacion en los campos 3D.

QUE MARCO USA CADA SALIDA — importa al leerla:

- `E_app`, `eps_app`, `sigma_fallo`, `frac_portante` y todo lo escalar son invariantes o ya estan referidos al eje ensayado: se leen tal cual.
- `campo_eps_eff` y `campo_vm` se devuelven en el marco ORIGINAL, listos para pintar sobre la geometria sin tocar nada.
- `eps`, `sig` y `u` se quedan en el marco DEL ENSAYO (donde la carga va en z). Deshacer esa rotacion componente a componente seria facil de equivocar y ninguna parte del visor los usa, asi que se marcan con `marco_permutado` en vez de convertirlos a medias.

`ensayo_triaxial(BW, spacing, ejes=(0, 1, 2), progreso=None, **kw)`

Ensayo de compresion en los tres ejes. Devuelve un dict por eje.

PARA QUE SIRVE: `E_app` de un solo eje no dice si la estructura es anisotropia; con los tres sale un cociente E_{max}/E_{min} que es una medida MECANICA de anisotropia, independiente del DA del tensor MIL —que es puramente geometrico— y comparable con E_x , E_y , E_z de la homogeneizacion periodica de `spinpy.elastic`.

Que las dos rutas coincidan no es trivial: la homogeneizacion impone contorno PERIODICO sobre la celda y este ensayo impone platos reales sobre una probeta finita. Coinciden solo si la estructura es grande frente a la trabecula. Discrepar es informacion —efecto de tamaño o de borde—, no un fallo; por eso conviene mirar las dos y no sustituir una por la otra.

La clave `resumen` trae `E_app` por eje y la anisotropia, ya calculados.

`estudio_convergencia(BW, spacing, resoluciones=(20, 24, 28, 32, 40), eje=2, remuestrear=None, progreso=None, **kw)`

`E_app` a varias resoluciones, para decidir si el valor es citable.

POR QUE HACE FALTA. En micro-elementos finitos sobre estructuras poco densas `E_app` NO converge sin más al refinar: medido sobre el VOI proximal de H4 (BV/TV 0.28) salieron 700, 797 y 365 MPa a 32^3 , 48^3 y 64^3 . Un valor tomado de una sola resolución no significa nada, y esa advertencia ya estaba escrita en la interfaz sin que hubiera forma de resolverla.

QUE SIGNIFICA UNA SERIE NO MONOTONA. No es ruido numérico —el problema es determinista y el residuo del solver se comprueba en cada punto—: es que al refinar cambia la GEOMETRIA. Cada remuestreo redibuja qué trabéculas quedan conectadas, y una trabécula de uno o dos vóxeles puede aparecer, desaparecer o partirse. El modelo no está convergiendo hacia una estructura fija; está resolviendo estructuras distintas. Por eso se devuelve también `rho` y `frac_portante` de cada punto: si varían, la comparación de `E_app` entre resoluciones no es una comparación de discretización.

Se devuelve además el orden de convergencia observado entre los tres últimos puntos y la extrapolación de Richardson, PERO solo cuando la serie es monótona y la densidad estable; en caso contrario se marcan como no aplicables en vez de dar un número que invitaría a citarlo.

`_4` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Mallas solidas: hexaedrica y TET10

`solido.py`

`solido.py` — Exportacion del spinodoide (o de un VOI) como SOLIDO.

Un STL es solo la piel: describe la frontera, no el interior. Para simular hace falta una malla VOLUMETRICA. Este modulo produce las dos variantes que tienen sentido sobre una mascara de voxeles.

HEXAEDRICA (directa desde los voxeles)

Cada voxel `solido` se convierte en un hexaedro de 8 nodos. Es exacta —reproduce la mascara sin ninguna aproximacion—, instantanea y NO PUEDE FALLAR: no hay mallador que se atragante. Es el metodo estandar de micro-elementos finitos para hueso trabecular (Van Rietbergen, Ruegsegger) y es lo que hacen `localWriteFEBioHex` y `localWriteAbaqusHex` de la app de MATLAB.

Su limitacion: el hexaedro trilineal es un elemento LINEAL, mas rigido a flexion que uno cuadratico con los mismos grados de libertad, y la superficie queda escalonada.

TETRAEDRICA TET10 (cuadratica)

Superficie por marching cubes, suavizada, reparada y tetraedralizada. La superficie sale lisa y el elemento cuadratico representa mejor la flexion, a cambio de ser mucho mas lento y de introducir tres pasos donde se pierde algo de material.

TRES COSAS QUE COSTARON DESCUBRIR Y QUE NO SE PUEDEN OMITIR

-
- 1 SUAVIZAR ES OBLIGATORIO antes de tetraedralizar. Sobre la superficie cruda de marching cubes tetgen aborta con `recoversubfaces` o `split_segment`: los datos binarios producen miles de triangulos coplanares y aristas diminutas que no puede recuperar. No es cosmetica.

- 1 EL ORDEN DE NODOS DE TETGEN NO ES EL DE C3D10. Medido (distancias exactamente cero):

ranura tetgen 4 5 6 7 8 9 arista (2,3) (0,3) (0,1) (1,2) (1,3) (0,2) C3D10 espera (0,1) (1,2) (0,2) (0,3) (1,3) (2,3)

De ahí `PERM_TETGEN_A_C3D10`. Sin la permutacion los elementos quedan retorcidos y el analisis corre igualmente devolviendo tensiones sin sentido: un error que no da ningun sintoma. Por eso se VERIFICA sobre la

mallla concreta antes de escribir.

- 1 SISTEMA DE UNIDADES mm-N-MPa. Ni Abaqus ni ANSYS llevan unidades: interpretan los numeros en el sistema coherente que elija quien escribe. La mallla esta en milimetros, asi que el modulo se escribe en MPa. Poner 20e9 (pascales) declara un material 10^6 veces mas rigido y el analisis corre sin protestar.

Funciones

`mallla_hex(BW, spacing)`

Un hexaedro por voxel solido. Devuelve (nodos, elems, informe).

Solo se crean los nodos que algun elemento usa: una mascara con BV/TV de 0.3 generaria de otro modo un 70% de nodos sueltos que ningun elemento referencia, y varios preprocesadores los rechazan.

`volumen_hex(nodos, elems)`

Volumen total, por el jacobiano en el centro de cada hexaedro.

Sirve de comprobacion independiente: si la conectividad estuviera en un orden equivocado los volumenes saldrian negativos.

`superficie_cerrada(BW, spacing)`

Isosuperficie 0.5 cerrada por relleno de vacio.

El relleno no es solo para cerrar: como en el corte el gradiente es puramente normal, las caras de los bordes salen PLANAS en vez de abiertas, que es lo que se necesita si despues hay que apoyar la probeta.

`mallla_tet10(BW, spacing, suavizado=SUAVIZADO_ITER, banda=SUAVIZADO_BANDA, decimado=DECIMADO, progreso=None)`

Mascara -> TET10. Devuelve (nodos, elems, superficie, informe).

`verificar_tet10(nodos, elems, tol_rel=1e-06, n_muestra=2000)`

Comprueba que cada nodo intermedio esta en el medio de SU arista.

1 funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica.

Exportadores: Abaqus, APDL, VTU, STL

`escribe.py`

`escribe.py` — Escritores de mallla para Abaqus, ANSYS, ParaView y STL.

SISTEMA DE UNIDADES: mm - N - MPa

Ni Abaqus ni ANSYS llevan unidades; interpretan los numeros en el sistema coherente que elija quien escribe el modelo. Las malllas de este proyecto estan en MILIMETROS, porque el spacing del micro-CT lo esta, y el sistema coherente con el milimetro es:

longitud mm fuerza N tension y modulo MPa = N/mm^2

Las funciones reciben E en PASCALES —como lo maneja el resto del proyecto, `localBaseMaterial` da 20e9— y lo escriben en MPa. Volcar 20e9 tal cual declara un material un millon de veces mas rigido, y el analisis corre sin protestar: solo se nota al ver desplazamientos absurdamente pequenos.

TIPOS DE ELEMENTO

hex8 Abaqus C3D8 ANSYS SOLID185 tet10 Abaqus C3D10 ANSYS SOLID187

Funciones

```
escribir_stl(superficie, ruta)
```

Sin documentar.

```
escribir_vtu(nodos, elems, ruta, arrays=True)
```

Malla volumetrica en VTU, para inspeccionarla en ParaView o pyvista.

Es el formato util para MIRAR la malla antes de mandarla a un solver: los .inp y .apdl son enormes y ningun visor los abre comodamente.

POR QUE LLEVA ARRAYS. Una malla sin ningun campo escalar es geometria muda: ParaView no puede colorearla ni construir una tabla de color, y al pedirle la representacion "Volume" responde

Failed to determine the LookupTable being used. Could not determine array range.

que suena a archivo corrupto y solo significa que no hay nada que mapear. Se incluyen por tanto la calidad y el volumen de cada elemento, que ademas son lo que de verdad interesa mirar: permiten localizar en pantalla los elementos degenerados antes de mandar la malla a un solver.

La calidad se evalua sobre la celda LINEAL —el hexaedro, o el tetraedro de las cuatro esquinas en el caso del TET10— porque el filtro de calidad de VTK no soporta celdas cuadraticas y devuelve -1 para todas, lo que aparenta una malla catastrofica cuando en realidad es que no sabe medirla. Como los TET10 son de lados rectos, la calidad de forma es la del tetraedro de esquinas y la medida es exacta, no una aproximacion.

```
escribir_abaqus(nodos, elems, ruta, E_s=200000000000.0, nu_s=0.3, nombre='SOLID0',  
conjuntos=True)
```

Malla volumetrica en formato Abaqus .inp.

ANSYS Mechanical lo importa directamente (External Model), asi que el mismo archivo sirve para las dos plataformas. Se escriben los conjuntos de nodos BASE y TECHO para que aplicar el apoyo y la carga en el preprocesador sea un clic y no una seleccion manual sobre cientos de miles de nodos.

```
escribir_apdl(nodos, elems, ruta, E_s=200000000000.0, nu_s=0.3)
```

Script de Mechanical APDL con NBLOCK/EBLOCK y conjuntos de nodos.

2 funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica.

Lote de VOIs y dispersion del generador

```
lote.py
```

lote.py — Ajuste y generacion de replicas en lote, con salida tabular.

PARA QUE SIRVE Y PARA QUE NO

El objetivo del proyecto es aumentar el N en estudios de volúmenes óseos escasos. Conviene ser preciso sobre que hace y que no hace este modulo, porque la diferencia decide si un resultado es publicable:

GENERAR K REPLICAS DE UN VOI NO DA $N = K$. Da $N = 1$ con K replicas tecnicas. La varianza total se descompone en la de ENTRE especimenes —la biologica, que fijan los animales— y la de DENTRO, que es el ruido del generador. Las replicas dividen la segunda por raiz de K y NO TOCAN la primera. Para una afirmacion sobre "el hueso sesamoideo equino" los grados de libertad los ponen los caballos, no las realizaciones.

Lo que las replicas SI compran:

- reducir la incertidumbre de la estimacion DE CADA especimen

- experimentos numericos controlados que el hueso real no permite (variar BV/TV con DA fijo, por ejemplo)
- rellenar huecos del muestreo
- alimentar un modelo sustituto estructura -> propiedad

Por eso la tabla que produce este modulo lleva SIEMPRE las columnas `animal`, `sitio` y `replica` separadas: sin esa estructura no se puede descomponer la varianza despues, y el error solo se detecta cuando ya se gastaron semanas de CPU. `spinpy.estadistica` opera directamente sobre ellas.

REPRODUCIBILIDAD

Cada fila registra la semilla y los parametros exactos que la produjeron, de modo que cualquier espécimen sintetico se puede regenerar bit a bit.

Funciones

```
dispersion_semillas(parametros, spacing, n_semillas=8, semilla_base=None, m_ref=None, extra=False, progreso=None)
```

Repite la MISMA parametrizacion con K semillas y resume la dispersion.

PARA QUE SIRVE. El generador es estocastico: dos evaluaciones del mismo vector de parametros dan realizaciones distintas. Esa dispersion es el SUELO DE RUIDO por debajo del cual ninguna diferencia significa nada — ni entre dos ajustes, ni entre Python y MATLAB, ni entre un candidato y el VOI. Toda la validacion del port se apoya en ella, y hasta ahora habia que cambiar la semilla a mano y anotar los numeros.

Si se pasa `m_ref` (tipicamente la morfometria del VOI), cada metrica trae ademas su `z`: cuantas desviaciones tipicas del propio generador separan al VOI de la media de las realizaciones. Es el mismo criterio con el que se valido el port — $|z| < 2$ es indistinguible del ruido— y convierte una diferencia porcentual, que no dice nada por si sola, en una afirmacion comprobable.

Un CV pequeno NO significa que la metrica sea fiable: significa que el generador la reproduce de forma estable. BS tiene un CV del 0.19% a densidad 0.55 y aun asi arrastra el sesgo de marching cubes.

```
correr_lote(vois, n_replicas=10, modo='completo', mecanica=False, res_homog=16, E_s=200000000000.0, nu_s=0.3, semilla_base=770000, n_jobs=-1, progreso=None)
```

Ajusta cada VOI y genera `n_replicas` sinteticas de cada ajuste.

Devuelve un DataFrame ordenado, una fila por espécimen:

origen 'real' para el VOI medido, 'sintetico' para las replicas animal H1..H4, C1..V5 — la unidad de inferencia biologica sitio proximal / medio / distal replica 0 para el real, 1..K para las sinteticas semilla la que genero esa realizacion (NaN en los reales) ... parametros ajustados y metricas

La fila real de cada VOI se incluye con `replica = 0` para que la comparacion real-vs-sintetico salga de la misma tabla y no de dos ficheros que puedan desincronizarse.

`_3` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Varianza, ICC, N efectivo y equivalencia TOST

`estadistica.py`

`estadistica.py` — Descomposicion de varianza y equivalencia real vs sintetico.

EL ERROR QUE ESTE MODULO EXISTE PARA IMPEDIR

Generar K replicas de cada VOI y despues tratarlas como K especimenes independientes es pseudorreplicacion (Hurlbert 1984). Infla los grados de libertad, estrecha artificialmente los intervalos de confianza y convierte cualquier diferencia en "significativa".

La varianza total se descompone en

$$\sigma^2_{\text{total}} = \sigma^2_{\text{entre}} + \sigma^2_{\text{dentro}}$$

donde `entre` es la variabilidad biologica entre especimenes y `dentro` la del generador. Promediar K replicas divide σ^2_{dentro} por K y deja σ^2_{entre} INTACTA. El N efectivo para una inferencia biologica lo fijan los especimenes, no las realizaciones.

`n_efectivo` lo calcula explicitamente para que aparezca en cualquier informe y sea dificil confundirlo con el numero de filas de la tabla.

POR QUE TOST Y NO UNA t DE STUDENT

Para sostener que los sinteticos SUSTITUYEN a los reales hace falta demostrar EQUIVALENCIA, y "no se rechaza la hipotesis nula" no es eso: con N pequeno no se rechaza nunca y con N grande se rechaza siempre, asi que la prueba clasica responde a la pregunta contraria a la que interesa.

TOST invierte la carga: se declara de antemano un margen y se demuestra que la diferencia cae dentro. Y ese margen no hay que inventarlo — el suelo de ruido del propio generador, ya medido en `Validacion_Anexo`, es el limite fisico por debajo del cual dos realizaciones de los MISMOS parametros tampoco se distinguen. Pedir mas seria pedir que el sintetico se parezca al real mas de lo que el real se parece a si mismo.

Funciones

```
descomponer_varianza(df, metrica, col_grupo='animal', col_replica='replica', solo=None)
```

Componentes de varianza entre y dentro de espécimen, ICC y N efectivo.

Modelo de efectos aleatorios de una via sobre los especimenes, con el estimador clasico de ANOVA (Searle):

$$\sigma^2_{\text{dentro}} = \text{CM}_{\text{dentro}} \quad \sigma^2_{\text{entre}} = (\text{CM}_{\text{entre}} - \text{CM}_{\text{dentro}}) / n_0$$

con n_0 el tamano medio armonico de grupo. σ^2_{entre} puede salir NEGATIVA cuando la variabilidad entre especimenes es menor que el ruido; se acota a cero y se marca, porque un valor negativo no es interpretable pero tampoco es un fallo del calculo.

```
descomponer_anidada(df, metrica, col_animal='animal', col_sitio='sitio',  
col_replica='replica', solo=None)
```

Descomposicion ANIDADA: animal / sitio(animal) / replica.

POR QUE NO BASTA UN SOLO NIVEL. Agrupar por `animal` mete en el mismo saco la variacion entre SITIOS del mismo animal y el ruido del generador, que son cosas distintas: la primera es biologia y la segunda es numerica. Sobre los VOIs equinos —4 animales x 3 sitios x K replicas— esa mezcla inflaba el CV "dentro" hasta duplicar el "entre" y hacia el ICC ininterpretable.

El diseno real tiene tres niveles y el modelo tiene que tener tres:

$$\sigma^2_{\text{total}} = \sigma^2_{\text{animal}} + \sigma^2_{\text{sitio}(\text{animal})} + \sigma^2_{\text{replica}}$$

ANOVA anidada clasica (Sokal & Rohlf) sobre un diseno balanceado. Los componentes pueden salir negativos cuando un nivel aporta menos que el ruido del siguiente; se acotan a cero y se marca, porque un valor negativo no es interpretable pero tampoco es un fallo de calculo.

El N para inferir entre animales sigue siendo el numero de ANIMALES.

```
tost(x, y, margen_rel, alfa=0.05)
```

Prueba de equivalencia por dos t unilaterales (Schuirmann).

Concluye equivalencia si el intervalo de confianza $(1-2*\alpha)$ de la diferencia de medias cae ENTERO dentro de $\pm$ margen. `margen_rel` es relativo a la media de `x` (el grupo de referencia), que es como se declara de forma natural a partir de un coeficiente de variación.

```
informe(df, metricas, margen_rel=None, col_grupo='animal')
```

Descompone la varianza y contrasta equivalencia real vs sintético.

`margen_rel` por métrica; si falta, se usa el CV DENTRO observado, que es el suelo de ruido del propio generador y por tanto el margen más exigente que tiene sentido pedir.

Linea de comandos

`cli.py`

`cli.py` — Entrada por línea de comandos, sin rutas propias de nadie.

POR QUE EXISTE. `correr_lote.py` sirve para trabajar, pero lleva el banco de VOIs escrito dentro:

BANCO = Path(r"C:\Users\carlo\OneDrive\Escritorio\VOIs Caballos")

Eso está bien para un script personal y es exactamente lo que impide que otra persona use la herramienta: la primera ejecución falla en una ruta que no existe en su máquina, y el mensaje no le dice que la cambie. Aquí las rutas son argumentos obligatorios.

`spinpy-lote VOI_*.vtk --replicas 20 --salida ./resultados`

El script antiguo se conserva: tiene los valores por defecto del banco de este proyecto y ahorra teclear. Los dos llaman al mismo `correr_lote`.

Funciones

```
main_lote(argv=None)
```

Sin documentar.

Interfaz grafica

`visor.py`

`visor.py` — Interfaz gráfica del port: VOI real y spinodoide candidato, lado a lado.

ALCANCE — menor que el de `AppFinal_V2`, y solo lo que está validado

Esta ventana expone lo que el núcleo portado respalda: generación del spinodoide, carga de un VOI, morfometría comparada, ajuste por búsqueda escalonada, tensor elástico homogeneizado, ensayo de compresión en uno o en los tres ejes con campos de deformación efectiva y de von Mises, y exportación como SOLIDO (malla hexaédrica o TET10) para Abaqus y ANSYS.

NO hay informes ni los otros cuatro optimizadores (Pareto, bayesiano, MOBO, best-fit rápido): esa lógica sigue viviendo solo en MATLAB. Un botón apagado sería peor que un botón ausente.

SESION REPRODUCIBLE

El menú Sesión guarda en JSON todo lo que determina el estado: parámetros, semilla, esquema de muestreo, resoluciones y la ruta del VOI. Como el generador es estocástico pero está sembrado, ese archivo regenera la MISMA realización bit a bit. Sin él, un caso interesante que se pierde al mover un deslizador no se recupera.

EXPORTAR: HEXAEDRICA O TET10

La hexaedrica pone un elemento por voxel. Es EXACTA —reproduce el 100% del volumen—, instantanea y no puede fallar al mallar, pero el elemento es lineal y la superficie queda escalonada.

La TET10 suaviza la superficie y la tetraedraliza: elemento cuadratico y piel lisa, a cambio de tiempo y de perder algo de material. Cuanto se pierde depende de la RESOLUCION: medido sobre el mismo spinodoide, 88.8% del volumen a 40^3 y 93.8% a 48^3 . Con trabeculas de uno o dos voxeles la superficie no tiene con que trabajar, asi que conviene exportar a la resolucion mas alta que se pueda pagar.

ORIENTACION IMPUESTA POR EL AJUSTE (correccion F4)

Los deslizadores de rotacion solo cubren ± 90 grados y no pueden representar todas las orientaciones de un VOI. Cuando el ajuste deriva una R del eje principal, esa R manda sobre los deslizadores y se avisa en el panel; mover cualquier rotacion la libera, porque ese gesto expresa que el usuario quiere controlar la orientacion a mano.

DOS RESOLUCIONES, QUE ES LO QUE HACE USABLE LA INTERFAZ

Generar y medir a 64^3 cuesta ~ 9 s en esta maquina, lo que vuelve inservible un deslizador. Aqui se separan:

- Resolucion de VISTA — se recalcula al mover un deslizador. A 32^3 con pocas ondas baja a decimas de segundo y sirve para explorar la forma.
- Resolucion de MEDIDA — solo se usa al pulsar "Medir". Es la que produce numeros comparables con el VOI, y es la que hay que citar.

Ambas usan la MISMA semilla, asi que muestrean el mismo campo aleatorio a distinta finura: la vista previa es una version gruesa de lo que se medira, no otra estructura.

El esquema de muestreo de ondas es un desplegable y no tiene valor por defecto silencioso: 'rechazo' (GIBBON/AppFinal_V2) y 'equitativo' (TPMS-Scaffolds-generator) producen anisotropias distintas para los mismos thetas cuando los conos tienen angulos desiguales. Ver README.

Uso: python visor.py

Funciones

`mallade_mascara(BW, spacing)`

Isosuperficie 0.5 de la mascara como PolyData.

Se construye un ImageData con la mascara en los NODOS y se contornea: es marching cubes de VTK, el mismo algoritmo que mide `area_superficie`, de modo que lo que se ve es lo que se mide.

`flatten(order='F')` porque VTK recorre X como indice mas rapido y BW tiene dimension $1 = X$ - la misma convencion que `readVTKVOI`.

Las normales se calculan EXPLICITAMENTE: `contour()` devuelve la malla sin ellas y, sin normales, `smooth_shading` no tiene con que sombrear. La estructura se renderiza entonces como una silueta plana de un solo color y parece un bloque macizo aunque tenga BV/TV de 0.28.

`main()`

`_Sin documentar._`

Clase Trabajador

Ejecuta una funcion lenta fuera del hilo de la interfaz.

Sin esto la ventana se congela durante la generacion y Windows la marca como 'no responde', que es justo lo que ocurre en la app de MATLAB con `drawnow` cuando la resolucion sube.

```
Trabajador.informar(self, i, n, etapa)
```

Callback de progreso para tareas largas.

Se invoca DESDE EL HILO DE TRABAJO. Emitir una senal es la forma correcta de cruzar al hilo de la interfaz: Qt la encola y la entrega en el bucle de eventos. Tocar los widgets directamente desde aqui provoca caidas intermitentes muy dificiles de reproducir.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `run`

Clase `DialogoFigura`

Base de los dialogos que muestran una figura con su lectura al pie.

Todos comparten la misma estructura —lienzo de matplotlib, tabla o nota en HTML debajo, y un boton para guardar la figura— y la misma regla: **ninguna figura se muestra sin el texto que dice como leerla**. Un histograma de espesor sin la advertencia del sesgo del 30%, o una curva de convergencia sin el veredicto, invitan a citar un numero que no es citable.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoEspesor`

Distribucion del espesor local, VOI y spinodoide superpuestos.

POR QUE UN HISTOGRAMA Y NO SOLO Tb.Th. El Tb.Th de la tabla es un escalar derivado de Parfitt ($2 \cdot BV/BS$): supone un modelo de placas y colapsa toda la estructura en un numero. Dos VOIs con el mismo Tb.Th pueden tener uno las trabeculas homogeneas y el otro una mezcla de trabeculas gruesas y finas —que es justo la firma de la perdida osea— y el escalar no los distingue. La distribucion si.

QUE NO SE PUEDE HACER CON ESTOS NUMEROS. La escala absoluta NO es citable como Tb.Th: a 3-4 voxeles de grosor el metodo de esferas inscritas subestima un $\sim 30\%$. Sirve para comparar dos estructuras medidas con el MISMO spacing y para leer la FORMA de la distribucion (moda, cola, bimodalidad). Los dos paneles comparten eje x por esa razon; con ejes distintos la superposicion no significaria nada.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoConvergencia`

E_app frente al tamaño de elemento, con el veredicto al pie.

La app ya advertia de que E_app no converge sin mas en estructuras poco densas, pero no ofrecia forma de comprobarlo. Esta figura es esa forma.

SE TRAZAN DOS COSAS EN EL MISMO EJE. Arriba E_app, que es lo que se quiere; abajo la **fraccion portante**, que es el diagnostico. Si la fraccion portante sube al refinar, la estructura conectada esta cambiando, y entonces la variacion de E_app no es error de discretizacion: son geometrias distintas. Sin esa segunda curva, una serie que sube y baja parece ruido numerico y se descarta por el motivo equivocado.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoDistribucionFE`

Distribucion de la deformacion efectiva y de la tension de von Mises.

POR QUE LA DISTRIBUCION Y NO EL MAXIMO. El criterio de Pistoia no mira el maximo —seria un solo elemento, y en una malla de voxeles ese elemento suele ser una esquina escalonada sin significado fisico—: mira el **percentil 98**, el valor que supera el 2% del tejido. Eso es un punto de la cola, y para saber si es robusto hay que ver la cola entera. Si esta concentrada en unos pocos nudos, el criterio depende de esos nudos; si esta repartida, el resultado

es estable.

Se traza en escala logaritmica en x: los dos campos abarcan varios ordenes de magnitud entre el material apenas cargado y los nudos, y en escala lineal todo se amontona contra el cero.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoDispersion`

Suelo de ruido del generador: K realizaciones de los mismos parametros.

LA PREGUNTA QUE CONTESTA. Cuando el candidato difiere del VOI en un 3% en Tb.Sp, ¿es que el ajuste falla o es que el generador no reproduce Tb.Sp mejor que eso? Sin esta medida no hay forma de saberlo, y la diferencia porcentual sola invita a interpretar ruido como senal.

La columna **z** es el criterio con el que se valido el port: cuantas desviaciones tipicas del propio generador separan al VOI de la media de las realizaciones. $|z| < 2$ significa indistinguible del ruido, y no se puede pedir a un ajuste que acierte mas de lo que el generador se repite a si mismo.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoVarianza`

Descomposicion de varianza del lote, con el N efectivo en primer plano.

EL ERROR QUE ESTE DIALOGO EXISTE PARA IMPEDIR. Generar K replicas de cada VOI y tratarlas despues como K especimenes independientes es pseudorreplicacion: infla los grados de libertad, estrecha los intervalos de confianza y convierte cualquier diferencia en significativa. El numero de filas de la tabla NO es el N del estudio.

Por eso el N efectivo va arriba y en grande, y no escondido en una columna.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `Deslizador`

Deslizador con etiqueta y valor, en unidades reales (no pasos enteros).

ALTURA MINIMA FIJA. Sin ella, cuando el panel no cabe a lo alto Qt reparte el deficit comprimiendo cada hijo, y la etiqueta acaba solapada sobre la barra: ilegible. Con un minimo, el panel prefiere desbordar —y entonces la QScrollArea que lo envuelve muestra una barra de desplazamiento— antes que aplastar los controles.

```
Deslizador.fijar(self, v)
```

Coloca un valor, acotandolo al rango del control.

Se acota en vez de fallar porque una sesion guardada con otra version puede traer un valor fuera de los limites de ahora; quedarse en el extremo y seguir es preferible a no cargar nada. Qt no emite `valueChanged` si el valor no cambia, asi que el aviso al usuario no puede colgar de la senal.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `valor`

Clase `Visor`

```
Visor.exportar_resultados(self)
```

Vuelca a CSV y JSON todo lo calculado en la sesion.

DOS ARCHIVOS, PORQUE SIRVEN PARA COSAS DISTINTAS. El CSV es plano y largo (una fila por magnitud) para pegarlo en una hoja o leerlo con pandas; el JSON conserva lo que no cabe en una tabla —el tensor C entero, los parametros, la traza— y es el que permite reproducir.

Se exporta lo que YA se calculo, sin recalculr nada. Con un generador estocastico, recalculr al exportar daria numeros distintos de los que el usuario vio en pantalla, y el archivo dejaria de ser el registro de lo ocurrido.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `histograma_espesor`, `cargar_voi`, `medir`, `ajustar`, `ensayo_fe`, `distribuciones_fe`, `convergencia_fe`, `exportar_solido`, `calcular_elastico`, `guardar_sesion`, `cargar_sesion`, `dispersion`, `correr_lote_gui`, `closeEvent`